Quickdress: Requirements Specification

Milestone 1 · Scoping, Requirements and Architecture · Revised November 5, 2026
Prepared by Omar Hassan
Team: Aya Noaman, Malk Metwally, Nada Salem, Marwan Abouelfotuh, Omar Hassan

What changed in this revision

This revision answers the instructor's comments on the first version (October 13, 2026). The architecture is now a microkernel (see the Design Specification), so the sentences that described the old layered design were updated. Statements that had no evidence or plan behind them were fixed:

- The comparison with Cladwell, Whering and Acloset (section 1.5) was wrong: Cladwell and Acloset already suggest daily outfits from the weather. It now cites their own descriptions and states what Quickdress does differently, plus how we will check it.
- The "10 minutes to 10 seconds" figure (section 1.2) had no source and was removed. The Sprint 2 usability test will time it instead.
- Stretch goals are no longer built "if capacity remains". Section 3.4 gives the rule that decides it, and each sprint has exit criteria.
- Social and environmental claims (section 5) now rely only on requirements that are in scope, or say clearly when a feature is a stretch goal or deferred.

Contents

1. System overview
2. Product backlog
3. Scope of the project
4. Use cases
5. Global, cultural, social, environmental, economic, public health, safety and welfare considerations
6. Conclusion
7. References

1 System overview

Quickdress is an iOS app that photographs a user's wardrobe once, then recommends a complete outfit from those clothes every morning, matched to the day's weather and calendar.

1.1 What it is

The user captures each clothing item with the phone camera. Quickdress stores the item with its attributes (type, colour, warmth, formality, comfort). Each morning the app presents one outfit for the day; the user can swap a single piece or request a new combination. Everything runs on the device: SwiftUI, SwiftData and a rule-based recommender. The app is built as a small core with plug-ins (see the Design Specification), so other recommenders can be added later without changing the screens. An optional step that asks Google's Gemini model for a suggestion is off by default and runs only if the user turns it on (FR-22).

1.2 Purpose and the problem it solves

Choosing clothes is a daily, repeated decision that costs time and causes decision fatigue. People repeat the same few outfits, forget items they own, and dress for the wrong weather or occasion. Quickdress removes the decision: the user confirms one suggested outfit made only from clothes they already own. We have not measured how long the choice takes today. The Sprint 2 usability test (NFR-04) will time five people choosing an outfit with and without the app.

1.3 Users

User	Need	How Quickdress serves it
University students	Little time in the morning, variable daily schedule (lectures, gym, events)	Calendar-aware outfit each morning
Young professionals	Dress codes that change by meeting; want to avoid repeating outfits in front of colleagues	Formality filter, repeat tracking
People with decision anxiety	Too many choices causes stress	One recommendation, not a catalogue
People unsure about colour matching (including colour-vision deficiency)	Cannot judge which colours go together	Colour matching handled by the recommender

1.4 Stakeholders

Stakeholder	Interest
End users (above)	A fast, trustworthy daily recommendation; privacy of wardrobe photos and calendar
Development team (Aya, Malk, Nada, Marwan, Omar)	Deliverable scope within the course timeline; learning Swift
Course instructor and TAs	Clear requirements, sound architecture, evidence of process (Jira, GitHub)
Apple (platform owner)	App Store guidelines, permission rules for camera, calendar and location
Weather data providers (Open-Meteo; Apple WeatherKit if a team member has an Apple Developer membership)	API usage within quota and terms
Google (Firebase AI Logic and the Gemini API)	Only if the opt-in AI step (FR-22) is built: API usage within quota and terms. It receives anonymised item attributes, never photos, calendar text or location.

1.5 Motivation

Quickdress is not the first app to suggest outfits from clothes the user owns. Cladwell's own guide says it sends three outfit suggestions based on the user's closet and the weather [1], and Acloset's App Store listing describes daily suggestions tailored to the weather and the user's schedule [2]. Whering is another digital wardrobe app in the same category. Quickdress differs in four design choices:

- Everything stays on the phone, with no account and no server of our own (NFR-02), and outfits are produced offline (NFR-03).
- Filters for the user's actual day: mode of transport (walking in the heat versus an air-conditioned car) and comfort priority (FR-10).
- Formality from the calendar that the user can correct, by tagging an event as formal or setting an override (FR-11).
- It is free, with no in-app purchases and no links to shops.

We have not found transport or comfort filters in these apps' public descriptions, but descriptions can be incomplete. In Sprint 1 the team will install Cladwell, Whering and Acloset, use each for one day, and record which of the four points above each app already covers. This section will be updated from that check.

The plan does not assume any prior iOS experience: the first sprint keeps to SwiftUI and SwiftData basics that Apple's own tutorials cover. Risk R3 in the Design Specification covers the rest of the learning curve.

2 Product backlog

The backlog lists everything the team has identified, including items that will not fit this term. Priority follows MoSCoW; Section 3 draws the implemented scope from the Must and Should rows.

2.1 Functional requirements

ID	Epic	Requirement	Priority
FR-01	Wardrobe capture	The user shall add a clothing item by taking a photo or choosing one from the photo library.	Must
FR-02	Wardrobe capture	The system shall store each item with attributes: category (top, bottom, outerwear, shoes, accessory), colour, warmth level, formality level, comfort level, and season.	Must
FR-03	Wardrobe capture	The user shall be able to edit or delete any stored item.	Must
FR-04	Wardrobe browsing	The user shall be able to view the wardrobe as a grid, filtered by category and colour.	Must
FR-05	Daily outfit	The system shall generate one complete outfit (top, bottom, shoes, plus outerwear when needed) each day from the user's own items.	Must
FR-06	Daily outfit	The user shall be able to replace any single piece in the recommended outfit with an alternative suggestion.	Must
FR-07	Daily outfit	The user shall be able to request a completely new outfit.	Must
FR-08	Weather	The system shall fetch the day's forecast for the user's location and use temperature and precipitation when choosing warmth level and outerwear.	Must
FR-09	Daily outfit	The user shall be able to mark an outfit as worn, and the system shall avoid repeating a worn outfit within a configurable number of days.	Should

ID	Epic	Requirement	Priority
FR-10	Custom filters	The user shall be able to set daily filters: mode of transport (walking, public transport, car), comfort priority, and formality override.	Should
FR-11	Calendar	The system shall read the day's events from the iOS Calendar (with permission) and raise the outfit's formality when an event is tagged formal, such as a presentation or interview.	Should
FR-12	Saved outfits	The user shall be able to compose and save their own outfits for the recommender to reuse.	Should
FR-13	Wardrobe capture	The system shall suggest the category and dominant colour of a photographed item automatically, which the user may correct.	Should
FR-14	Daily outfit	The system shall send a morning notification at a user-chosen time with the day's outfit.	Could
FR-15	Insights	The system shall show which items have not been worn in the last 30 and 90 days.	Could
FR-16	Daily outfit	The user shall be able to rate a recommended outfit, and the recommender shall learn from ratings.	Could
FR-17	Wardrobe capture	The system shall remove the photo background so items display consistently.	Could
FR-18	Travel	The user shall be able to generate a packing list for a trip of N days at a destination.	Won't (this term)
FR-19	Social	The user shall be able to share an outfit image with friends.	Won't (this term)
FR-20	Laundry	The system shall track items currently in the laundry and exclude them from recommendations.	Won't (this term)
FR-21	Multi-device	The wardrobe shall sync across the user's devices via iCloud.	Won't (this term)
FR-22	AI suggestions	The user shall be able to turn on AI suggestions. When they are on, the system may ask an external model (Gemini) for an outfit, sending only anonymised item attributes and the day's conditions.	Could

2.2 Non-functional requirements

ID	Quality	Requirement	Priority
NFR-01	Performance	The daily outfit screen shall be visible within 2 seconds of opening the app on an iPhone 12 or newer.	Must
NFR-02	Privacy	Wardrobe photos, item data and calendar information shall be stored and processed on the device only; no personal data leaves the phone without explicit user consent.	Must
NFR-03	Availability (offline)	Wardrobe browsing and outfit generation shall work with no network connection, using the last fetched forecast.	Must
NFR-04	Usability	A new user shall be able to add their first item and receive an outfit within 5 minutes, without instructions.	Must
NFR-05	Performance	Generating an outfit for a wardrobe of up to 200 items shall take under 1 second.	Should

ID	Quality	Requirement	Priority
NFR-06	Reliability	The app shall not lose any stored item or photo across app updates or force-quits.	Should
NFR-07	Security	Access to the Camera, Photos, Calendar and Location shall each be requested only when first needed, with an explanation, and the app shall remain usable if any permission is denied.	Should
NFR-08	Maintainability	The recommendation engine shall be behind a single interface so the on-device model can be swapped for an API-based model without changing the UI.	Should
NFR-09	Accessibility	All screens shall support VoiceOver labels and Dynamic Type up to the Accessibility XL size.	Could
NFR-10	Compatibility	The app shall support iOS 17 and later.	Could
NFR-11	Localisation	UI strings shall be externalised so Arabic can be added without code changes.	Could

3 Scope of the project

The team commits to the 11 functional requirements FR-01 to FR-11 and the 8 non-functional requirements NFR-01 to NFR-08. FR-12, FR-13 and FR-22 are stretch goals, built in Sprint 3 only under the decision rule in section 3.4. Everything else in the backlog is deferred.

3.1 Functional requirements in scope

Group	Requirements	Why in scope
Wardrobe capture and browsing	FR-01, FR-02, FR-03, FR-04	Without a stored wardrobe nothing else works; this is the foundation.
Daily outfit	FR-05, FR-06, FR-07	The core promise of the product: open the app, see today's outfit.
Weather	FR-08	The first custom filter and the simplest external integration (Open-Meteo needs no key; WeatherKit is used if the team has a developer membership).
Context filters	FR-09, FR-10, FR-11	These are the differentiators named in the pitch: no repeats, transport and comfort filters, calendar awareness.
Stretch	FR-12, FR-13, FR-22	Saved outfits, automatic attribute suggestion and the opt-in Gemini step improve the experience, but the app is complete without them. Built only under the decision rule in section 3.4.

3.2 Non-functional requirements in scope

NFR-01 to NFR-08: performance (2 s to first screen, 1 s to generate), on-device privacy, offline operation, 5-minute first-run usability, data durability, permission handling, and a swappable recommendation engine. NFR-09 to NFR-11 (accessibility, iOS version floor, localisation) are followed as good practice but not formally tested this term.

3.3 Out of scope this term

FR-14 to FR-21: notifications, wear insights, rating-based learning, background removal, travel packing, social sharing, laundry tracking and iCloud sync. Each needs either a server, extra ML work, or an additional Apple framework, and none is required to prove the concept.

3.4 Why the scope is achievable

The scope is split into three sprints, one per remaining course milestone. Each sprint ends at that milestone with a working app that can be demonstrated, and counts as done only when its exit criteria pass (Design Specification, Table 8). The riskiest work, the outfit engine, is in Sprint 2, so Sprint 3 still leaves room to recover.

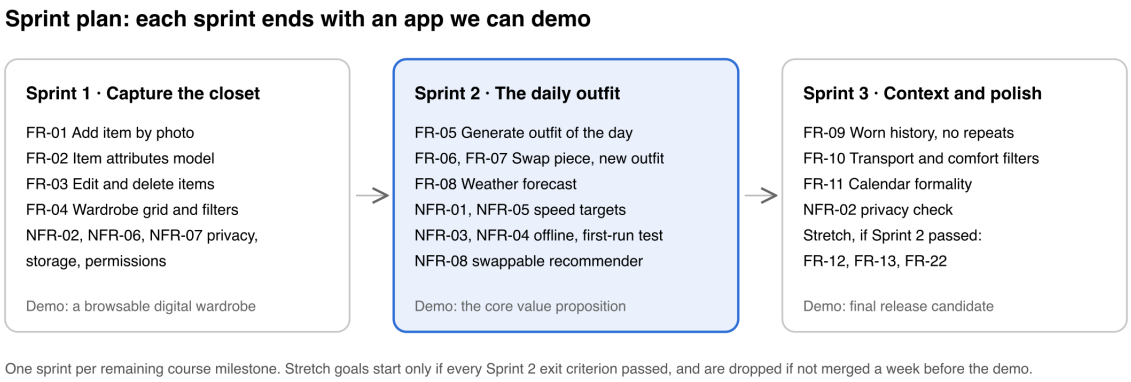


Figure 1. Sprint plan: three sprints, one per remaining milestone.

Sprint 1 is mostly SwiftUI and SwiftData create, read, update and delete work, which suits a team that is learning Swift. Sprint 2 is the outfit engine. Its recommender is rule-based scoring over item attributes, with every rule and weight written out in section 2.2 of the Design Specification, not a trained model, which keeps machine learning off the critical path. Sprint 3 adds the calendar and two more filters, each a small plug-in.

Stretch goals follow one rule. They are started only if every Sprint 2 exit criterion has passed when Sprint 3 begins. They are then built in the order FR-12, FR-13, FR-22, and any stretch goal that is not merged one week before the Sprint 3 demo is dropped.

The plan is tied to the course milestones, not to calendar dates, because the team has not fixed sprint dates yet. When each sprint starts, its start and end dates are entered on the Jira board.

4 Use cases

Quickdress has one primary actor, the User, and three secondary actors that the system calls: a weather service (Open-Meteo, or Apple WeatherKit if available), the iOS Calendar (EventKit) and, only when the user has turned on AI suggestions, Google's Gemini model through Firebase AI Logic. The diagram covers the in-scope requirements FR-01 to FR-11 and the stretch requirement FR-22.

Quickdress: use case diagram

One primary actor drives every use case. Weather, calendar and the opt-in Gemini step are secondary actors the system calls.

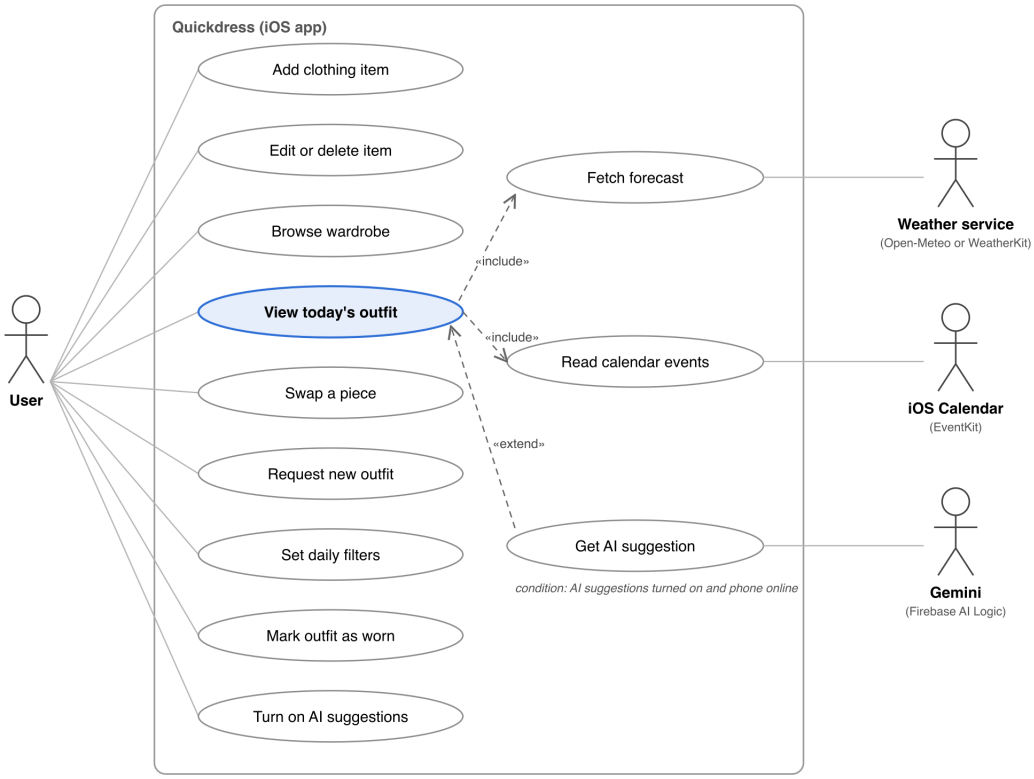


Figure 2. Use case diagram: one primary actor, three secondary actors, twelve use cases.

Viewing today's outfit always includes fetching the forecast and reading calendar events; if either permission is denied the use case still completes with defaults (NFR-07). Getting an AI suggestion extends it only when the user has turned the feature on and the phone is online.

4.1 Use case descriptions

Use case	Actor	Precondition	Main flow	Postcondition	Requirements
Add clothing item	User	Camera or Photos permission granted	1. User taps Add. 2. Takes or picks a photo. 3. Sets category, colour, warmth, formality, comfort, season. 4. Saves.	Item stored locally with photo and attributes	FR-01, FR-02
Edit or delete item	User	Item exists	1. User opens an item. 2. Changes attributes or taps Delete. 3. Confirms.	Item updated or removed; outfits containing it are invalidated	FR-03
Browse wardrobe	User	At least one item	1. User opens Wardrobe. 2. Filters by category or colour. 3. Taps an item for detail.	None	FR-04
View today's outfit	User	At least one top, bottom	1. App opens on Today. 2. System fetches forecast and calendar (included).	Outfit of the day displayed and cached	FR-05, FR-08, FR-11, NFR-01

Use case	Actor	Precondition	Main flow	Postcondition	Requirements
		and pair of shoes stored	3. Recommender scores combinations and shows the best one.		
Swap a piece	User	Outfit displayed	1. User taps a piece. 2. System offers the next best alternatives for that slot. 3. User picks one.	Outfit updated	FR-06
Request new outfit	User	Outfit displayed	1. User taps New outfit. 2. System generates the next best full combination.	Outfit replaced	FR-07
Set daily filters	User	None	1. User opens Filters. 2. Sets transport mode, comfort priority, formality override. 3. Applies.	Recommendation regenerated with the filters	FR-10
Mark outfit as worn	User	Outfit displayed	1. User taps Wearing this.	Outfit logged with today's date; excluded from recommendations for the configured period	FR-09
Turn on AI suggestions	User	None	1. User opens Settings. 2. Reads what data would be sent. 3. Turns on AI suggestions.	Consent stored; the Gemini step may run when online	FR-22, NFR-02
Fetch forecast	System (weather service)	Location permission granted, network available	1. System requests today's hourly forecast. 2. Stores it locally.	Forecast cached; last forecast used when offline	FR-08, NFR-03
Read calendar events	System (EventKit)	Calendar permission granted	1. System reads today's events. 2. Detects formal keywords or user tags.	Formality level for the day set	FR-11
Get AI suggestion	System (Gemini)	AI suggestions on and network available	1. System sends anonymised item attributes and the day's conditions. 2. Gemini returns one item per slot. 3. System checks the reply against the wardrobe.	Outfit returned; on any failure the rule-based recommender answers instead	FR-22, NFR-02, NFR-03

5 Global, cultural, social, environmental, economic, public health, safety and welfare considerations

The most significant consideration is privacy: Quickdress holds photographs of a person's clothes, their location and their calendar. Processing everything on the device (NFR-02) is a design decision made because of this, not only for performance.

Area	Consideration	How the requirements respond
Privacy and security	Wardrobe photos, calendar entries and location reveal daily routine and whereabouts.	On-device storage and inference (NFR-02); permissions requested one at a time with an explanation and the app usable without them (NFR-07); no account, no server of our own, no analytics. The only external AI call (FR-22) is off by default and sends anonymised attributes only.
Public health and safety	Dressing for the wrong weather causes discomfort and, in Cairo summers, heat-related risk; morning decision stress is a small daily wellbeing cost.	Weather-driven warmth and outerwear choice (FR-08); one recommendation instead of a catalogue (FR-05) to reduce decision fatigue.
Cultural and social	Dress norms differ by religion, workplace and family; modest dress, hijab-compatible combinations and local occasions (Eid, Ramadan iftars, university events) are common for the target users. The recommender must not impose a single Western style norm.	Formality and comfort are user-defined attributes, not inferred from fashion datasets (FR-02); if the stretch goal FR-12 is built, the user can also save their own outfits as examples; no body-shape or appearance scoring anywhere in the backlog.
Environmental	Fast fashion is driven partly by people feeling they have nothing to wear.	Quickdress recommends only from what the user owns and favours items that have not been worn recently (the freshness score, Design Specification section 2.2), which encourages re-wearing instead of buying. A screen that lists unworn items (FR-15) is deferred.
Economic	Target users are students and early-career professionals with limited budgets.	The app is free, has no in-app purchases, and never links to shopping. Development cost is low: first-party Apple frameworks and no server hosting. Weather comes from the free Open-Meteo API, with WeatherKit used only if a team member already has the paid Apple Developer membership, and the opt-in Gemini step uses Google's lowest-cost model.
Global and accessibility	The first release is English-only and iOS-only, which limits reach in Egypt where Android dominates. Colour matching is a real barrier for users with colour-vision deficiency.	Strings externalised for Arabic (NFR-11); VoiceOver and Dynamic Type (NFR-09); colour matching handled by the system so the user never has to judge colours. Android is noted as a future direction, not a requirement.
Public welfare	A recommender that nudges users towards particular clothes could be exploited for advertising.	NFR-02 rules out third-party data sharing without consent, and the scope contains no monetisation feature or commercial recommendations.

6 Conclusion

Quickdress solves a small but daily problem, choosing what to wear, by recommending one outfit from the user's own wardrobe, matched to the weather and the day's calendar. This document fixes the Milestone 1 baseline: a backlog of 22 functional and 11 non-functional requirements, a committed scope of FR-01 to FR-11 and NFR-01 to NFR-08 with FR-12, FR-13 and FR-22 as stretch goals, twelve use cases, and a three-sprint plan in which each sprint ends with a

demonstrable app. The accompanying Design Specification shows how the microkernel architecture, a small core with plug-ins for capture, storage, context, filters and recommenders, meets the non-functional requirements committed here, and how each one will be tested. The next delivery is Sprint 1: a working digital wardrobe with capture, editing and browsing.

7 References

[1] Cladwell, "Getting started with Cladwell". https://cladwell.com/blog/getting-started (accessed November 5, 2026).

[2] Acloset, "Acloset - AI Fashion Assistant", Apple App Store listing. https://apps.apple.com/us/app/acloset-ai-fashion-assistant/id1542311809 (accessed November 5, 2026).